

RL-1: Steering GPT-2 with PPO

Data Mining — Deep Learning, Bonus Challenge (25 pts)

Sobhan Moghimi - Amirhossein Azarpoor

Instructor: Dr. Hadi Farahani

Term: Spring 2026

Abstract

This report describes a small RLHF-style experiment: GPT-2 is fine-tuned with proximal policy optimization (PPO) so that continuations of IMDB review prompts become more positive, using a frozen DistilBERT sentiment classifier as the reward. The original contribution is a controlled comparison of training *with* versus *without* a KL penalty to a frozen reference model. Both runs raise the classifier score, but the no-KL policy drifts much farther from GPT-2 (KL ≈ 0.83 vs. 0.20) and is about $2.5\times$ less fluent under frozen-GPT-2 perplexity (225 vs. 89). Qualitative samples show the same gap: with KL the text still looks like a review; without KL it collapses into praise keywords and factually broken sentences.

1 Problem and data

The handbook task is to start from GPT-2 and use PPO so that continuations of movie-review prompts are steered toward a target sentiment. The scalar reward is the sentiment classifier’s score on the generated text. This is a miniature version of RLHF, the method used to align chat models [3, 4].

Data. Prompts come from the IMDB movie-review set (`stanfordnlp/imdb` on the Hugging Face Hub) [2]. Following the standard TRL sentiment tutorial [6], I keep reviews longer than 200 characters and take a random prefix of 8–16 tokens as the prompt. A subset of 6000 prompts is used so the Colab run finishes in a reasonable time. The policy never sees gold continuations: it only sees the prompt and later a scalar reward.

Reward model. The off-the-shelf classifier `lvwerra/distilbert-imdb` scores the concatenation of prompt and continuation. I use the raw `POSITIVE` logit, not the 0/1 label, because a smoother signal is easier for PPO. A quick sanity check on two sentences gave $\approx +2.72$ for “This movie was wonderful.” and ≈ -2.94 for “This movie was terrible.”

2 Method

Policy and reference. The backbone is GPT-2 small, already language-model fine-tuned on IMDB (`lvwerra/gpt2-imdb`), wrapped with TRL’s `AutoModelForCausalLMWithValueHead` so PPO has a learned value function. A frozen copy of the same weights is the reference policy π_{ref} .

PPO objective. At each step the policy π_θ generates a continuation y given prompt x . The classifier returns a score $r(x, y)$. The token-level reward used by PPO is the usual KL-shaped form from the TRL / Ziegler setup:

$$R_t = -\beta \text{KL}(\pi_\theta(\cdot \mid x, y_{<t}) \parallel \pi_{\text{ref}}(\cdot \mid x, y_{<t})) + \mathbf{1}[t = T] r(x, y), \quad (1)$$

where β is the KL coefficient and the classifier score is applied on the last generated token. Advantages are estimated with GAE ($\gamma = 1$, $\lambda = 0.95$). The policy is then updated with the clipped PPO surrogate plus a value-function loss.

Current TRL (≥ 0.12) replaced the old `PPOTrainer.step(queries, responses, rewards)` loop with a `HuggingFaceTrainer.train()` API that scores a causal-LM reward model on GPT-2 token ids. That API cannot use DistilBERT (different tokenizer). I therefore kept TRL’s value-head model and implemented the classic generate $\rightarrow$ DistilBERT reward $\rightarrow$ PPO update loop, which is the algorithm the assignment actually needs.

Training details. Two independent runs, same seed (42), 60 PPO steps, batch size 16, Adam learning rate 1.41×10^{-5} , 4 PPO epochs per batch, clip range 0.2, response length 24 tokens, sampling with $top_k = 0$, $top_p = 1.0$.

- **with KL:** $\beta = 0.2$ (the classic TRL default);
- **no KL:** $\beta = 0.0$.

Fluency is measured as perplexity of 40 samples under a *frozen* vanilla GPT-2, not under the tuned policy, so the number is a fair “does this still look like English?” check.

3 Original contribution

Reproducing the TRL GPT-2 sentiment tutorial is not enough [1]. My contribution is the first option listed on the project card: a **reward-hacking study** that trains with and without the KL-to-reference penalty and reports what happens to fluency as reward climbs.

Hypothesis. If PPO is only maximizing classifier score, the policy can exploit quirks of DistilBERT (repeated “wonderful”, adjective lists, etc.) and leave the GPT-2 distribution. A KL penalty should cap that drift: slightly lower reward, but generations that still look like movie reviews.

This is a controlled ablation around a stated hypothesis, not a generic “A vs B”.

4 Experiments

Both runs start from the same checkpoint and see the same prompt pool. The only changed knob is β . Metrics:

- mean POSITIVE logit per PPO step (reward);
- mean $\text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ per step;
- frozen-GPT-2 perplexity on 40 generations after training;
- side-by-side qualitative continuations on five held-in prompts.

I did not add a supervised baseline (fine-tune on labeled positive reviews) or train my own reward model; those are listed as alternative / stretch contributions. The KL ablation already answers the required report items: reward curve, before/after generations, fluency-vs-reward, and what the KL term changed.

5 Results and analysis

5.1 Reward curves

Figure 1 (left) shows that *both* policies learn: the POSITIVE logit starts around 0–0.5 and trends upward. The no-KL run reaches the higher peaks (about 2.5). That matches the hypothesis — without a KL cost the optimizer can chase the classifier more aggressively. The last no-KL point falls to 0.64; with batch size 16 that is mostly noise, so I treat the *trend and peaks* as the reward result, not that single step.

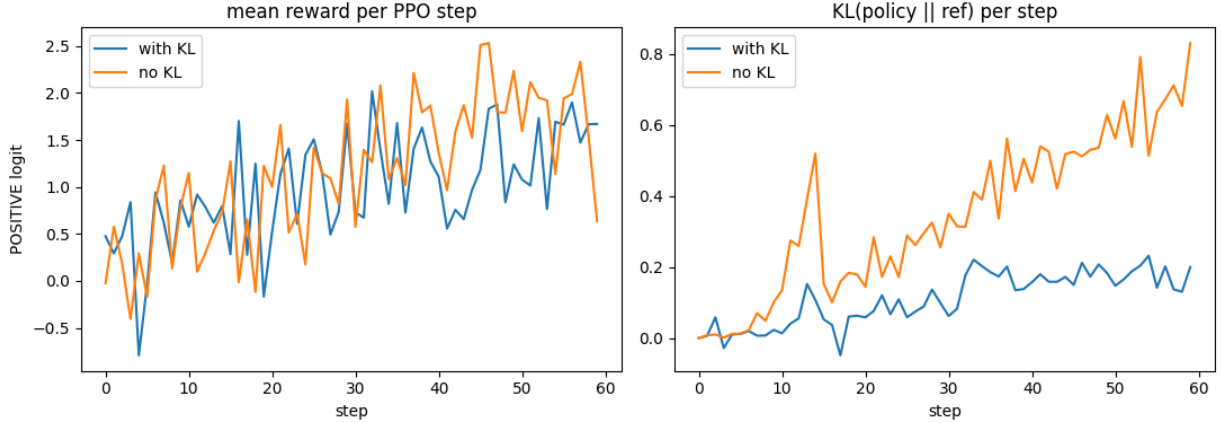


Figure 1: Left: mean DistilBERT POSITIVE logit per PPO step. Right: KL between the current policy and the frozen reference. Blue = $\beta = 0.2$, orange = $\beta = 0$.

Table 1: Final numbers after 60 PPO steps. Reward/KL are the last-batch means; perplexity is computed on 40 samples under frozen GPT-2.

Run	Final reward (logit)	Final KL($\pi \parallel \pi_{\text{ref}}$)	Perplexity $\downarrow$
with KL ($\beta = 0.2$)	1.67	0.20	89.3
no KL ($\beta = 0$)	0.64*	0.83	225.1

*The last no-KL batch dropped; the orange curve peaks near ≈ 2.5 around steps 45–57. Perplexity is the more stable fluency summary.

5.2 What the KL penalty changed

Figure 1 (right) is the cleaner plot. With $\beta = 0.2$, KL stays in a band of roughly 0.1–0.2. With $\beta = 0$ it climbs almost linearly to 0.83. The no-KL policy is leaving the reference GPT-2 distribution. Frozen-GPT-2 perplexity agrees: 89 with KL vs. 225 without (about $2.5\times$ worse). So the KL term is doing what it is supposed to do: it buys fluency by holding π_θ near π_{ref} , at the cost of a more conservative reward.

5.3 Qualitative generations

Table 2 shows five prompts. A few observations:

- **Steering works.** Prompt 3 starts negative (“disinterest kills *The Psychic*”) and the +KL model still continues with “brilliantly executed ... a truly memorable film.” That is the actual task succeeding.
- **+KL stays review-like.** Continuations are grammatical and mostly on-topic (DVD, “most amazing film of the last 20 years”). There is still some GPT-2 clumsiness (“wonderful and wonderful”); 60 steps on GPT-2 small will not sound like a human critic.
- **No-KL reward-hacks.** The same prompts become adjective lists: “Love Lovely musical comedy. Wonderful storyline. Love songs”; “beautifully and beautifully”. Prompt 5 is the clearest failure: Robert Bloch, the *author* of *Psycho*, is described as “wonderful as Dr. Watson’s wife ... the ideal husband.” DistilBERT likes *wonderful* / *beautiful* / *love*; PPO piles those words on even when they contradict the prompt.

This is the fluency-vs-reward trade-off in text form. No-KL looks “more positive” to the classifier and worse to a reader.

Table 2: Side-by-side continuations (prompt in italics). +KL is $\beta = 0.2$; -KL is $\beta = 0$.

Generation	
<i>I think this is almost all I need to say. I feel obliged</i>	
+KL	... to tell you ... that I think this is the most amazing film of the last 20 years by David
-KL	... to share my joy ... with many wonderful performances and exciting performances; unforgettable songs and great performances
<i>Allow me to just get to the bottom line here: I've got 3 kids</i>	
+KL	... they are ALL 12 years old, and that's exactly the type of music I would choose to listen to
-KL	... their parents loved the movie. They loved it. Love Lovely musical comedy. Wonderful storyline. Love songs
<i>A story of amazing disinterest kills "The Psychic" over and over again</i>	
+KL	... so brilliantly executed by writer/director Sam Harris. A truly memorable film.
-KL	... Some scenes make us feel so deeply ... The songs sound haunting, powerful and beautifully performed.
<i>REALLY??? I</i>	
+KL	... did watch the film about 15 years later and even got it on DVD
-KL	... like Karen Bates's portrayals ... beautifully and beautifully; a wonderfully diverse portrayal of love
<i>The late, great Robert Bloch (author of PSYCHO, for</i>	
+KL	... wrote a wonderful and wonderful book about the early 1930's
-KL	... is wonderful as Dr. Watson's wife. He is beautiful. He plays the ideal husband; caring, compassionate

6 Discussion and observation

The two questions below are copied from the project card [1].

Question 1

Supervised fine-tuning optimizes cross-entropy against fixed target text; PPO optimizes expected reward from a classifier with no fixed targets at all. What fundamentally different training signal does each provide, and why can pure reward maximization drift into fluent-looking gibberish or degenerate repetition without a KL penalty holding the policy near a reference model?

Answer. SFT and PPO are not two ways of doing the same thing with a different optimizer. They provide different *kinds* of supervision.

SFT is token-level imitation. For a pair (x, y^*) written by a human, the loss is

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_t \log \pi_\theta(y_t^* | x, y_{<t}^*).$$

Every update is pulled toward a specific string that existed in the data. If the model starts saying “great great great”, that continuation is simply not in y^* , so the gradient pushes it back toward real reviews.

PPO never sees a target string. After generating $y \sim \pi_\theta(\cdot | x)$ it only receives a scalar $r(x, y)$ from DistilBERT. The training signal is “make this number go up.” Any y that the classifier likes is a valid solution: a coherent positive review, but also “Love Lovely musical comedy. Wonderful storyline. Love songs”, or calling Robert Bloch “Dr. Watson’s wife” as long as the words *wonderful* / *beautiful* / *caring* are there. The classifier was trained to distinguish positive vs. negative IMDB text; it was not trained to reject fluent-looking nonsense. So the set $\{y : r(x, y) \text{ is large}\}$ is much bigger than the set of texts a human would write.

Without a KL term, nothing in the loss cares whether y still looks like language from π_{ref} . The optimizer is free to walk into a region of output space that exploits the reward model. In my run that region is visible in three independent measurements at once: (i) $\text{KL}(\pi \| \pi_{\text{ref}})$ grows to 0.83 instead of staying near 0.20; (ii) frozen-GPT-2 perplexity jumps from 89 to 225; (iii) the samples become repetitive

buzzwords. The KL penalty $\beta \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$ puts a cost on that walk. It does not tell the policy *what* to say; it only says “do not leave the original GPT-2 neighborhood while you chase r .” That is why +KL still writes review-like sentences and why no-KL can look positive to DistilBERT while looking broken to a person.

Question 2

This setup is a miniature of how real chat models are aligned with RLHF. In that setting, what plays the role of the sentiment classifier, and what could go wrong — for the model’s behavior — if that reward model is systematically wrong about what it’s rewarding?

Answer. In this toy, DistilBERT stands in for “how positive is this review?” In production RLHF the same slot is filled by a *learned reward model* (RM), usually a Transformer trained on human preference pairs: raters see two assistant replies and pick which one is better (more helpful, less harmful, etc.) [4, 5]. PPO then treats the RM score the way I treated the POSITIVE logit.

If that RM is systematically wrong, the policy will optimize the error, not the intended goal, because PPO never sees the rater — it only sees the number. Concrete failure modes, by analogy with this experiment:

- **Proxy exploitation / reward hacking.** DistilBERT likes the word *wonderful*, so no-KL repeats it. A chat RM that overweights polite, confident tone will produce sycophantic or verbose answers that look “helpful” on the RM and are not actually useful.
- **Blind spots become policy.** If raters (or the RM) miss a class of harm — subtle factual errors, jailbreak-style compliance, bias — PPO will not penalize it and may even amplify it if those completions score well on the other axes the RM does track.
- **Goodhart’s law.** Once the policy is optimized against a fixed RM, the RM score stops being a good measure of the real objective. My no-KL model has a high classifier score and a high perplexity; a chat model can have a high RM score and still hallucinate, refuse too much, or flatter the user.

That is why real pipelines keep a KL (or similar) penalty to the SFT / reference model, and why the RM itself is refreshed with new human data. The KL term is not just a fluency trick: it is a brake on exploiting a misspecified reward.

7 Limitations and next steps

This is a short Colab run, not a full RLHF training job.

- **Compute.** 60 steps $\times$ batch 16 on GPT-2 small. Reward curves are noisy; a longer run would make the no-KL last-step dip less relevant.
- **One reward model.** I used `lvwerra/distilbert-imdb` as given. The stretch goal — labeling preference pairs and training a tiny RM myself — is not done.
- **No SFT baseline.** A supervised fine-tune on classifier-labeled positive reviews would make the Q1 contrast even sharper (fixed targets vs. scalar reward). I only compared two PPO settings.
- **GPT-2 artifacts remain.** Even +KL is sometimes off-topic (kids / music on a movie prompt) and still has mild repetition. That is the backbone, not only PPO.

If I continued, I would log perplexity every 10 steps (reward vs. fluency as a curve, not just two end points) and add a short SFT-on-positive-reviews baseline.

8 Conclusion

PPO can steer GPT-2 continuations toward positive sentiment using DistilBERT as reward. The original contribution was to turn the KL coefficient off and show the cost. Reward still goes up — often higher without KL — but the policy leaves the reference model (KL 0.83 vs. 0.20), perplexity more than doubles (225 vs. 89), and the text degenerates into classifier-friendly gibberish. The KL penalty is what keeps this miniature RLHF setup from collapsing into reward hacking.

References

- [1] H. Farahani. *Data Mining Deep Learning Project Handbook*, Spring 2026. Project card RL-1.
- [2] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng, and C. Potts. Learning word vectors for sentiment analysis. In *ACL*, 2011. Dataset: `stanfordnlp/imdb`.
- [3] D. M. Ziegler, N. Stiennon, J. Wu, T. B. Brown, A. Radford, D. Amodei, P. Christiano, and G. Irving. Fine-tuning language models from human preferences. `arXiv:1909.08593`, 2019.
- [4] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. Christiano. Learning to summarize with human feedback. In *NeurIPS*, 2020.
- [5] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, et al. Training language models to follow instructions with human feedback. In *NeurIPS*, 2022.
- [6] L. von Werra, Y. Belkada, L. Tunstall, E. Beeching, T. Thrush, N. Lambert, and S. Huang. TRL: Transformer Reinforcement Learning. <https://github.com/huggingface/trl>. Models: `lvwerra/gpt2-imdb`, `lvwerra/distilbert-imdb`.